INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DE SANTA CATARINA

CÂMPUS SÃO JOSÉ

WAGNER FLORES DOS SANTOS

ANÁLISE COMPARATIVA ENTRE ARQUITETURAS DE AGENTE ÚNICO E MULTIAGENTE BASEADAS EM LLM PARA CHATBOTS DE ATENDIMENTO

SÃO JOSÉ

2026

Wagner Flores dos Santos

Análise Comparativa entre Arquiteturas de Agente Único e Multiagente Baseadas em LLM para Chatbots de Atendimento

Pré-projeto apresentado à Unidade Curricular de Trabalho de Conclusão de Curso I do Curso de Engenharia de Telecomunicações do Instituto Federal de Educação, Ciência e Tecnologia de Santa Catarina – Câmpus São José.

Área de concentração: Telecomunicações.

Orientador: Prof. Arliones Stevert Hoeller Júnior, Dr.

São José

2026

RESUMO

Os modelos de linguagem em grande escala (*Large Language Models* – LLMs) ampliaram as possibilidades de automação do atendimento ao cliente, permitindo interpretar solicitações, consultar informações e produzir respostas mais flexíveis do que as obtidas com fluxos baseados apenas em regras. Uma das abordagens exploradas consiste em distribuir as etapas do atendimento entre agentes especializados, coordenados para executar funções distintas. Ainda não está claro, porém, em quais condições essa distribuição de responsabilidades oferece vantagens suficientes para compensar o aumento da complexidade e do consumo de recursos em relação a uma arquitetura de agente único. Este trabalho tem como objetivo comparar experimentalmente as duas arquiteturas em um cenário controlado de atendimento textual de um provedor de Internet, considerando a correção e a relevância das respostas, a conclusão das tarefas, a precisão dos encaminhamentos, o número de chamadas ao modelo e o consumo de tokens. Para isso, serão implementados dois protótipos, submetidos ao mesmo conjunto de casos de teste e às mesmas condições de execução, com avaliação apoiada pelo DeepEval e por métricas registradas diretamente durante o experimento. Espera-se identificar em quais situações a especialização por agentes setoriais melhora o atendimento e quais efeitos essa organização produz sobre o número de chamadas ao modelo, o uso de tokens e o custo operacional.

Palavras-chave: modelos de linguagem em grande escala; sistemas multiagentes; chatbots; atendimento ao cliente; agentes inteligentes.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura proposta de agente único 24
Figura 2 – Arquitetura multiagente proposta 27
Figura 3 – Fluxo de processamento de uma mensagem 32

LISTA DE QUADROS

Quadro 1 – Síntese dos trabalhos relacionados 20
Quadro 2 – Correspondência entre objetivos específicos, etapas e entregas previstas 34

LISTA DE TABELAS

Tabela 1 – Distribuição prevista dos cenários conversacionais 31
Tabela 2 – Exemplo de cenário conversacional 33
Tabela 3 – Estrutura prevista para apresentação dos resultados 37
Tabela 4 – Cronograma previsto para o TCC2 no primeiro semestre de 2027 . . . 39

LISTA DE CÓDIGOS

Código 3.1 – Exemplo ilustrativo de retorno do agente único 25
Código 3.2 – Exemplo ilustrativo de retorno da triagem 28
Código 3.3 – Exemplo ilustrativo de solicitação de nova triagem 29

LISTA DE ABREVIATURAS E SIGLAS

API *Application Programming Interface.*

JSON *JavaScript Object Notation.*

LLM *Large Language Model.*

SUMÁRIO

1 INTRODUÇÃO 10
1.1 PROBLEMA DE PESQUISA 11
1.2 JUSTIFICATIVA 11
1.3 OBJETIVOS 12
1.3.1 Objetivo geral 12
1.3.2 Objetivos específicos 12
1.4 DELIMITAÇÃO DO TRABALHO 13
1.5 ORGANIZAÇÃO DO TEXTO 13

2 REVISÃO BIBLIOGRÁFICA 14
2.1 FUNDAMENTAÇÃO TEÓRICA 14
2.1.1 Modelos de linguagem em grande escala 14
2.1.2 Agentes baseados em modelos de linguagem 15
2.1.3 Arquiteturas de agente único e multiagente 15
2.1.4 Comunicação, orquestração e retenção de contexto 16
2.1.5 Avaliação de sistemas conversacionais 17
2.2 TRABALHOS RELACIONADOS 18
2.3 SÍNTESE DA REVISÃO E LACUNA DE PESQUISA 20

3 METODOLOGIA 22
3.1 NATUREZA DO TRABALHO 22
3.2 DELINEAMENTO EXPERIMENTAL 22
3.2.1 Arquiteturas avaliadas 23
3.2.1.1 Arquitetura de agente único 23
3.2.1.2 Arquitetura multiagente 26
3.2.2 Variáveis controladas 29
3.2.3 Variáveis avaliadas 30
3.3 BASE DE CONHECIMENTO E PROMPTS 30
3.4 CENÁRIOS CONVERSACIONAIS 31
3.4.1 Cenário conversacional de exemplo 31
3.5 MATERIAIS E FERRAMENTAS 33
3.6 PROCEDIMENTO EXPERIMENTAL 34
3.7 MÉTRICAS E CRITÉRIOS DE AVALIAÇÃO 35
3.8 APRESENTAÇÃO PREVISTA DOS RESULTADOS 36
3.9 PRIVACIDADE E PROTEÇÃO DOS DADOS 37
3.10 RISCOS E LIMITAÇÕES 37

4 CRONOGRAMA DE ATIVIDADES 39

Referências 40

1 INTRODUÇÃO

Os *Large Language Model*s (LLMs) ampliaram as possibilidades de automação de atividades que envolvem compreensão e geração de linguagem natural. Quando integrados a bases de conhecimento, ferramentas e mecanismos de decisão, esses modelos podem formar agentes capazes de interpretar solicitações, manter o contexto de uma conversa e executar tarefas com algum grau de autonomia. No atendimento ao cliente, essa abordagem permite desenvolver soluções mais flexíveis do que os chatbots baseados apenas em regras e fluxos predefinidos.

Trabalhos recentes apresentam aplicações de LLMs na classificação de chamados e na geração de respostas de suporte (MONCADA et al., 2025), bem como arquiteturas formadas por agentes coordenadores e especializados para atuar em diferentes componentes de sistemas de telecomunicações (LEE; TSENG; CHEN, 2025). Essas soluções podem ser organizadas em arquiteturas de agente único ou multiagente.

Na arquitetura de agente único, um mesmo componente interpreta as mensagens, consulta as informações disponíveis, decide o procedimento e produz as respostas. Essa centralização reduz a necessidade de coordenação entre componentes. Na arquitetura multiagente, essas responsabilidades são distribuídas entre agentes com funções específicas, como triagem, primeiro atendimento, comercial, ouvidoria e suporte técnico. A divisão pode facilitar a especialização e reduzir o contexto recebido por cada agente, mas também introduz etapas adicionais de comunicação e novos pontos de falha (YAN et al., 2026).

A comunicação entre os agentes influencia a eficiência, a escalabilidade e a robustez do sistema. Uma decisão incorreta na triagem pode encaminhar a conversa para um agente que não possui as informações necessárias. Também podem ocorrer perdas de contexto, encaminhamentos desnecessários ou propagação de erros entre os componentes. Falhas de agentes e interrupções na comunicação também são apontadas como riscos desses sistemas (OWOTOGBE, 2025).

Estudos recentes investigam o encaminhamento de conversas para atendimento humano e o reconhecimento de intenções em ambientes multiagentes (AMRI et al., 2026; YANG; XU; ZHANG, 2025). Os resultados apontam benefícios da especialização, mas também mostram que a vantagem da arquitetura multiagente não ocorre em todas as métricas. Além disso, os trabalhos adotam diferentes modelos, cenários e critérios de avaliação, o que dificulta uma comparação direta entre arquiteturas de agente único e multiagente.

Esses resultados indicam que a adoção de uma arquitetura multiagente não representa, necessariamente, uma melhoria em todas as situações. Ainda são necessárias

comparações nas quais as arquiteturas utilizem o mesmo modelo, a mesma base de conhecimento, parâmetros equivalentes e cenários de atendimento comparáveis. Também é necessário observar o desenvolvimento da conversa, as decisões de encaminhamento e os recursos utilizados até a conclusão da tarefa.

1.1 PROBLEMA DE PESQUISA

A escolha entre uma arquitetura de agente único e uma arquitetura multiagente envolve um compromisso entre qualidade, complexidade e custo operacional. A especialização pode melhorar o tratamento de determinadas solicitações, mas exige coordenação entre agentes e pode aumentar o número de chamadas ao modelo e o consumo de tokens.

Uma avaliação baseada somente em perguntas isoladas não representa todo o funcionamento de um chatbot de atendimento. Em uma conversa real, as informações são fornecidas gradualmente, e uma resposta pode depender das mensagens anteriores. O sistema também pode precisar rever o encaminhamento quando o usuário apresenta uma nova informação ou muda de assunto durante o atendimento.

Diante desse contexto, este trabalho busca responder à seguinte pergunta:

> Quais diferenças de desempenho são observadas entre uma arquitetura de agente único e uma arquitetura multiagente, ambas baseadas em modelos de linguagem em grande escala e aplicadas a cenários conversacionais de atendimento, quanto à correção e à relevância das respostas, à conclusão das tarefas, à precisão dos encaminhamentos, ao número de chamadas ao modelo e ao consumo de tokens?

1.2 JUSTIFICATIVA

A escolha da arquitetura influencia tanto a qualidade do atendimento quanto o custo de operação de sistemas baseados em LLMs. Uma solução centralizada pode ser suficiente para tarefas simples e utilizar menos chamadas ao modelo. Por outro lado, a distribuição das responsabilidades entre agentes especializados pode favorecer situações que exigem regras específicas ou mudanças de setor durante uma conversa, embora acrescente etapas de comunicação e novos pontos de falha (YAN et al., 2026).

No atendimento de provedores de Internet, uma mesma conversa pode envolver dúvidas gerais, solicitações comerciais, reclamações e problemas técnicos. Um encaminhamento incorreto pode levar o usuário para uma fila inadequada ou produzir uma resposta sem as informações necessárias. Estudos anteriores demonstram que a especialização pode melhorar determinadas métricas, mas essa vantagem não ocorre em todas

as situações. Também foi observada uma tendência de o sistema multiagente realizar transferências para atendimento humano mais cedo (AMRI et al., 2026).

A contribuição esperada deste trabalho é a implementação de dois protótipos equivalentes e a realização de uma comparação controlada entre as arquiteturas. A avaliação considerará a qualidade das respostas, a conclusão das tarefas, os encaminhamentos realizados e os recursos consumidos. Os resultados poderão servir como referência para pesquisadores e desenvolvedores na escolha de uma arquitetura compatível com os requisitos de qualidade, organização e custo da aplicação.

1.3 OBJETIVOS

1.3.1 Objetivo geral

Comparar experimentalmente o desempenho de uma arquitetura de agente único e de uma arquitetura multiagente, ambas baseadas em modelos de linguagem em grande escala e aplicadas a cenários conversacionais de atendimento textual de um provedor de Internet.

1.3.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- definir uma base de conhecimento e regras de atendimento comuns às duas arquiteturas;
- especificar e implementar um protótipo de agente único e um protótipo multiagente composto por um agente de triagem e agentes especializados;
- elaborar e revisar um conjunto de cenários conversacionais de teste para primeiro atendimento, comercial, ouvidoria e suporte técnico;
- avaliar a correção e a relevância das respostas, a conclusão das tarefas e a precisão dos encaminhamentos;
- medir o consumo de tokens e o número de chamadas ao modelo durante cada conversa;
- estimar o custo de execução de cada arquitetura;
- comparar os resultados para identificar as vantagens, limitações e custos de cada arquitetura.

1.4 DELIMITAÇÃO DO TRABALHO

O estudo será restrito ao atendimento textual e utilizará cenários sintéticos baseados em situações típicas de atendimento de um provedor de Internet. Todos os dados empregados serão fictícios, sem utilização de conversas ou cadastros de clientes reais e sem interação com clientes durante o experimento.

As duas arquiteturas utilizarão o mesmo modelo de linguagem, os mesmos parâmetros de geração e a mesma base de conhecimento. No agente único, as regras e informações serão reunidas em um único contexto. No sistema multiagente, o conteúdo será distribuído conforme as responsabilidades da triagem e dos agentes especializados.

Serão consideradas quatro áreas de atendimento: primeiro atendimento, comercial, ouvidoria e suporte técnico. Os cenários poderão conter mudanças de assunto durante a conversa, permitindo verificar o comportamento das arquiteturas diante da necessidade de um novo encaminhamento.

Não fazem parte do escopo o atendimento por voz, o processamento de conteúdo multimodal, o treinamento ou ajuste fino de modelos, a comparação entre diferentes LLMs e a implantação do sistema com clientes reais. Também não serão realizados testes de robustez com injeção deliberada de falhas nos agentes ou na comunicação.

1.5 ORGANIZAÇÃO DO TEXTO

Além deste capítulo introdutório, o trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, os trabalhos relacionados e a lacuna de pesquisa; o Capítulo 3 descreve as arquiteturas propostas, a base de conhecimento, os cenários conversacionais, as ferramentas, o procedimento experimental e os critérios de avaliação; e o Capítulo 4 apresenta o cronograma das atividades previstas para o desenvolvimento do trabalho no TCC2.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta os conceitos utilizados no desenvolvimento do trabalho e discute pesquisas relacionadas ao uso de modelos de linguagem em sistemas de atendimento. São abordados os agentes baseados em modelos de linguagem, as arquiteturas de agente único e multiagente, a comunicação entre agentes e os critérios empregados na avaliação de sistemas conversacionais.

2.1 FUNDAMENTAÇÃO TEÓRICA

2.1.1 Modelos de linguagem em grande escala

Os modelos de linguagem em grande escala são capazes de interpretar instruções e produzir textos a partir do contexto fornecido em sua entrada. Em sistemas de atendimento, esses modelos podem ser utilizados para identificar a intenção do usuário, consultar informações e elaborar respostas adequadas à solicitação recebida.

Diferentemente de um fluxo baseado somente em regras fixas, um sistema apoiado em LLMs pode tratar diferentes formas de expressar uma mesma intenção. Essa característica é útil em conversas nas quais o usuário não segue um roteiro predefinido ou fornece as informações de maneira incompleta.

O comportamento do modelo depende das instruções, das informações disponíveis e do histórico incluído no contexto. Uma resposta também pode variar entre execuções, mesmo quando a entrada é semelhante. Por esse motivo, a utilização de LLMs em processos de atendimento exige critérios que permitam verificar a correção das respostas e o cumprimento das regras definidas para o serviço.

As informações processadas pelo modelo são representadas por tokens, que correspondem a unidades de texto utilizadas na entrada e na saída. A quantidade de tokens depende do tamanho dos prompts, do histórico da conversa, da base de conhecimento fornecida e das respostas produzidas. Como arquiteturas multiagentes podem realizar mais de uma chamada ao modelo para tratar uma mesma mensagem, o número de chamadas e o consumo de tokens são medidas relevantes para a comparação proposta.

Yang, Xu e Zhang observam que modelos de linguagem podem melhorar o reconhecimento de intenções em comparação com métodos baseados somente em regras ou classificação tradicional. Os autores também destacam o custo computacional e a latência associados ao seu uso (YANG; XU; ZHANG, 2025). Embora a latência não seja uma variável avaliada neste trabalho, o estudo demonstra que a utilização de LLMs acrescenta custos operacionais que devem ser considerados na escolha da arquitetura.

2.1.2 Agentes baseados em modelos de linguagem

Um agente baseado em modelo de linguagem combina o modelo com instruções que definem sua função, informações de contexto e mecanismos para executar ações. Em um chatbot de atendimento, essas ações podem incluir classificar uma solicitação, consultar uma base de informações, formular uma resposta ou encaminhar a conversa para outro setor.

As instruções fornecidas ao agente delimitam sua atuação. Um agente de suporte técnico, por exemplo, pode receber regras sobre os testes permitidos, os dados que devem ser solicitados e as situações que exigem transferência para atendimento humano. O histórico da conversa permite utilizar informações fornecidas em mensagens anteriores e evitar que o usuário tenha que repeti-las.

Agentes baseados em modelos de linguagem também podem utilizar mecanismos de memória e recuperação de informações. Yan et al. (2026) destacam que técnicas de geração aumentada por recuperação podem fornecer ao modelo informações obtidas de fontes externas ou de memórias estruturadas. Nesse tipo de abordagem, os conteúdos relevantes são recuperados dinamicamente e inseridos no contexto utilizado para produzir a resposta.

Neste trabalho não será implementado um mecanismo de geração aumentada por recuperação. A base de conhecimento será formada por documentos previamente definidos, contendo regras, informações e procedimentos fictícios de atendimento. No agente único, todo o conteúdo será disponibilizado em um contexto comum. Na arquitetura multiagente, o mesmo conteúdo será dividido conforme a responsabilidade de cada agente.

Moncada et al. utilizam agentes especializados para realizar a ingestão de dados, a classificação de chamados e a geração de respostas em uma aplicação de suporte ao cliente (MONCADA et al., 2025). O trabalho mostra que uma tarefa de atendimento pode ser dividida entre componentes com responsabilidades distintas. Essa divisão, entretanto, exige que as informações necessárias sejam transmitidas corretamente entre os agentes.

2.1.3 Arquiteturas de agente único e multiagente

Na arquitetura de agente único, um mesmo agente concentra as funções necessárias ao atendimento. Ele recebe as mensagens, interpreta a solicitação, mantém o histórico, consulta as informações disponíveis e produz a resposta. As regras dos diferentes setores ficam reunidas no contexto desse agente.

Essa organização reduz a necessidade de encaminhamentos internos. Por outro lado, o aumento da quantidade de regras e assuntos pode tornar o contexto mais extenso e dificultar a separação das responsabilidades. O agente precisa identificar quais instruções devem ser aplicadas em cada situação.

Na arquitetura multiagente, as responsabilidades são divididas entre agentes com funções específicas. Um agente pode realizar a triagem da conversa, enquanto outros tratam de assuntos comerciais, técnicos ou administrativos. Lee, Tseng e Chen apresentam uma organização semelhante para sistemas de suporte à operação de telecomunicações, composta por um agente principal e agentes especializados (LEE; TSENG; CHEN, 2025).

A especialização permite limitar as instruções e informações recebidas por cada agente. Em contrapartida, torna necessário definir como o sistema seleciona o próximo agente e quais partes do histórico serão transferidas. Assim, parte do desempenho da arquitetura passa a depender do mecanismo de coordenação.

Amri et al. comparam arquiteturas de agente único e multiagente no encaminhamento de conversas para atendimento humano (AMRI et al., 2026). Os resultados apresentados pelos autores indicam diferenças entre as arquiteturas, mas não demonstram vantagem da abordagem multiagente em todos os critérios. Essa constatação reforça a necessidade de comparar as duas alternativas sob as mesmas condições.

Yan et al. (2026) classificam as arquiteturas de comunicação de sistemas multiagentes baseados em modelos de linguagem em estruturas planas, hierárquicas, por equipes, de sociedade e híbridas. Nas estruturas hierárquicas, componentes de nível superior coordenam ou direcionam a execução de agentes responsáveis por tarefas específicas. Essa organização facilita a divisão das responsabilidades, mas acrescenta etapas de comunicação e pode concentrar decisões em determinados componentes.

A arquitetura multiagente proposta neste trabalho aproxima-se de uma organização hierárquica com coordenação centralizada. Um orquestrador será responsável por manter o estado da conversa e controlar a execução. O agente de triagem selecionará o primeiro agente especializado. Enquanto o assunto permanecer na mesma área, as mensagens serão encaminhadas ao agente ativo. Quando esse agente identificar uma mudança de assunto, solicitará uma nova triagem, sem escolher diretamente o próximo especialista.

2.1.4 Comunicação, orquestração e retenção de contexto

A orquestração corresponde ao controle do fluxo de execução entre os componentes do sistema. Em uma arquitetura multiagente de atendimento, o orquestrador recebe a mensagem, recupera o estado da conversa e aciona a triagem ou o agente especializado que se encontra ativo. O orquestrador não produz a resposta de atendimento, mas controla quais componentes serão executados e registra as decisões tomadas.

A triagem é responsável por classificar o assunto e indicar o agente adequado. Depois dessa decisão, o agente especializado permanece ativo enquanto as mensagens continuarem relacionadas à sua área. Caso identifique uma mudança de assunto, ele retorna ao orquestrador uma solicitação de nova triagem. Dessa forma, os especialistas

não realizam transferências diretas entre si.

Em uma conversa com vários turnos, as informações costumam ser fornecidas gradualmente. O usuário pode informar inicialmente que está sem acesso à Internet e somente depois relatar o estado das luzes do equipamento. Se houver troca de agente entre essas mensagens, o novo agente precisa receber as informações necessárias para continuar o atendimento.

Para representar esse comportamento, o sistema deverá manter o histórico das mensagens, os fatos relevantes extraídos da conversa e a identificação do agente ativo. O histórico preservará as mensagens trocadas; os fatos relevantes poderão registrar dados como CEP, condição de cliente e problema relatado; e a identificação do agente ativo indicará qual componente deve receber a mensagem seguinte.

A comunicação entre os agentes não ocorrerá de maneira livre. As trocas serão intermediadas pelo orquestrador e utilizarão saídas estruturadas, contendo a intenção identificada, o destino indicado, a ação solicitada e os fatos que devem permanecer no contexto. Essa estrutura permitirá registrar o caminho percorrido por cada conversa e verificar em qual etapa ocorreu um possível erro.

Yan et al. (2026) analisam os sistemas multiagentes a partir da comunicação estabelecida entre seus componentes. Segundo essa perspectiva, não basta avaliar separadamente a capacidade de cada agente, pois o resultado também depende das mensagens trocadas, do conteúdo transferido e das decisões tomadas durante a execução.

Falhas em um componente podem afetar as etapas seguintes de um sistema multiagente. Owotogbe (2025) discute esse problema ao investigar a robustez de sistemas formados por agentes baseados em modelos de linguagem. Embora a injeção deliberada de falhas não faça parte do escopo deste trabalho, a possibilidade de propagação de erros justifica o registro do fluxo percorrido por cada atendimento.

2.1.5 Avaliação de sistemas conversacionais

A avaliação de um sistema conversacional pode considerar tanto a resposta entregue ao usuário quanto o processo utilizado para produzi-la. Uma resposta pode estar correta, mas ter sido obtida após encaminhamentos desnecessários, repetição de perguntas ou uso excessivo de chamadas ao modelo.

A correção indica se a resposta respeita as informações e regras disponíveis. A relevância verifica se o conteúdo produzido trata diretamente da solicitação apresentada. A retenção de conhecimento observa se informações fornecidas em turnos anteriores permanecem disponíveis durante a conversa. A conclusão da tarefa verifica se as necessidades definidas para o cenário foram atendidas.

Neste trabalho, parte da avaliação será realizada com o DeepEval, uma biblioteca

destinada à avaliação de aplicações baseadas em modelos de linguagem (CONFIDENT AI, 2026). Para a avaliação de conversas com múltiplos turnos, as mensagens do usuário e do assistente são organizadas como um caso de teste conversacional. Sobre esse caso podem ser aplicadas métricas que utilizam um modelo de linguagem como avaliador.

Entre as métricas inicialmente previstas estão a relevância dos turnos, a retenção de conhecimento e a completude da conversa (CONFIDENT AI, 2026). A relevância dos turnos verifica se cada resposta permanece relacionada ao contexto anterior. A retenção de conhecimento observa se o sistema preserva fatos informados pelo usuário. A completude verifica se as intenções apresentadas durante a conversa foram atendidas. O DeepEval fornece, para cada métrica, uma pontuação e uma justificativa produzida pelo modelo avaliador.

O DeepEval também disponibiliza recursos para apoiar a geração de casos e a simulação de usuários. No entanto, os cenários deste trabalho serão definidos e revisados pelo pesquisador. A ferramenta poderá auxiliar na produção de variações de redação e na execução de conversas, mas nenhum caso gerado automaticamente será incorporado ao experimento sem conferência prévia.

Nem todos os resultados serão avaliados por um modelo. Informações objetivas, como a fila selecionada, o número de chamadas, o consumo de tokens e a ocorrência de uma transferência, serão verificadas diretamente nos registros da execução. Essa combinação reduz a dependência de avaliações subjetivas produzidas por outro LLM.

Como a avaliação baseada em modelo também pode apresentar variações ou interpretações inadequadas, as duas arquiteturas serão analisadas com o mesmo modelo avaliador, as mesmas configurações e os mesmos critérios. Uma amostra das avaliações automáticas também será conferida manualmente.

2.2 TRABALHOS RELACIONADOS

Os trabalhos analisados abordam diferentes aspectos do uso de modelos de linguagem e sistemas multiagentes. Alguns apresentam aplicações voltadas ao atendimento, enquanto outros tratam do reconhecimento de intenções, da comunicação entre agentes e da robustez dessas arquiteturas.

Moncada et al. (2025) apresentam uma arquitetura multiagente para automação de chamados de suporte. A solução foi implementada com o *framework* CrewAI e distribui as tarefas entre agentes responsáveis pela entrada dos dados, classificação dos chamados e geração das respostas. Os autores relatam resultados de acurácia na classificação e de similaridade semântica entre as respostas produzidas e as respostas esperadas. O trabalho demonstra uma aplicação prática da especialização de agentes, mas não realiza uma comparação controlada com uma arquitetura de agente único nem acompanha conversas

com múltiplos turnos.

No contexto das telecomunicações, Lee, Tseng e Chen (2025) propõem uma arquitetura para sistemas de suporte à operação. A organização é composta por um agente principal, responsável pela coordenação, e agentes específicos, encarregados de tarefas relacionadas a diferentes áreas do sistema. Os autores apontam benefícios relacionados à flexibilidade e à divisão de responsabilidades. Entretanto, o estudo concentra-se na descrição da arquitetura e não apresenta uma comparação experimental entre agente único e multiagente utilizando os mesmos cenários e parâmetros.

O reconhecimento de intenções é investigado por Yang, Xu e Zhang (2025). Os autores propõem uma abordagem em duas etapas. Na primeira, utilizam TF-IDF (*Term Frequency–Inverse Document Frequency*), uma técnica que atribui pesos às palavras conforme sua frequência em um texto e sua raridade no conjunto de documentos. A mensagem do usuário e os nomes e descrições dos agentes são representados por vetores, comparados por similaridade de cosseno para selecionar o agente adequado. Quando a similaridade não atinge um limiar predefinido, um modelo de linguagem é acionado para identificar a intenção. A proposta procura equilibrar acurácia, latência e custo computacional. O estudo é pertinente ao mecanismo de triagem, mas sua avaliação concentra-se na identificação da intenção, sem avaliar a continuidade do atendimento após o encaminhamento.

Amri et al. (2026) comparam arquiteturas de agente único e multiagente na decisão de encaminhar uma conversa para atendimento humano. Para realizar os testes, os autores utilizam uma base formada por diálogos de atendimento e comparam o desempenho das arquiteturas com diferentes modelos de linguagem. Os resultados indicam melhor desempenho do sistema multiagente em parte das medidas avaliadas. Entretanto, também mostram que a organização em múltiplos agentes pode alterar o momento em que ocorre a transferência. O estudo é próximo da proposta deste trabalho, mas está concentrado na decisão de transferência para humanos.

A comunicação em sistemas multiagentes baseados em modelos de linguagem é analisada por Yan et al. (2026). Os autores organizam as arquiteturas a partir dos participantes, dos tipos de mensagens, das estruturas de comunicação e das estratégias utilizadas para coordenar as tarefas. A revisão também discute problemas como aumento do custo de comunicação, propagação de erros e dificuldade de manter informações consistentes entre os agentes. Por se tratar de uma revisão, o trabalho não realiza uma comparação experimental própria, mas fornece a base conceitual para analisar a comunicação entre os componentes.

A robustez dos sistemas multiagentes é discutida por Owotogbe (2025), que propõe o uso de princípios de engenharia do caos para avaliar o comportamento do sistema diante de falhas. O trabalho considera que problemas em um agente ou na comunicação podem afetar o resultado produzido pelos demais componentes. Embora este TCC não

inclua a injeção deliberada de falhas, essa discussão mostra a importância de registrar os encaminhamentos e observar em qual etapa ocorre um resultado incorreto.

O Quadro 1 resume as principais características dos trabalhos analisados e sua relação com a proposta desta pesquisa.

Quadro 1 – Síntese dos trabalhos relacionados

Trabalho	**Aplicação**	**Abordagem**	**Limitação em relação a este TCC**
Moncada et al. (2025)	Chamados de suporte	Agentes para entrada dos dados, classificação e geração de respostas	Não compara as duas arquiteturas nem avalia conversas com múltiplos turnos
Lee, Tseng e Chen (2025)	Operações de telecomunicações	Agente principal coordenando agentes especializados	Não apresenta comparação experimental controlada entre agente único e multiagente
Yang, Xu e Zhang (2025)	Reconhecimento de intenção	Classificação inicial com TF-IDF e uso de LLM como segunda etapa	Avalia a identificação da intenção, mas não a continuidade do atendimento
Amri et al. (2026)	Transferência para atendimento humano	Comparação entre agente único e multiagente	Concentra-se na decisão de transferência para atendimento humano
Yan et al. (2026)	Comunicação entre agentes	Revisão de arquiteturas, mensagens e estratégias de comunicação	Não realiza comparação experimental própria
Owotogbe (2025)	Robustez de sistemas multiagentes	Aplicação de princípios de engenharia do caos	Possui procedimento e objetivo experimental diferentes

Fonte: Elaborada pelo autor.

2.3 SÍNTESE DA REVISÃO E LACUNA DE PESQUISA

Os trabalhos analisados mostram que a divisão de responsabilidades entre agentes pode ser aplicada à classificação de solicitações, ao encaminhamento de conversas e à execução de tarefas especializadas. Também indicam que a comunicação entre os componentes pode aumentar a complexidade do sistema e introduzir problemas relacionados à latência, ao consumo de recursos e à perda de informações.

Entre os estudos selecionados, o trabalho de Amri et al. (2026) é o que mais se aproxima da comparação proposta neste TCC. Entretanto, sua avaliação está direcionada ao encaminhamento para atendimento humano. Os demais estudos analisam aplicações ou aspectos específicos das arquiteturas, sem comparar agente único e multiagente durante conversas que possam envolver diferentes setores.

Permanece, portanto, a necessidade de uma comparação na qual as duas arquiteturas utilizem o mesmo modelo de linguagem, a mesma base de conhecimento, parâmetros

equivalentes e os mesmos cenários de atendimento. Além da resposta final, essa comparação deve observar a retenção das informações fornecidas ao longo da conversa, as decisões de encaminhamento e os recursos utilizados por cada arquitetura.

Com base nessa lacuna, este trabalho propõe implementar dois protótipos equivalentes e submetê-los a cenários conversacionais de atendimento de um provedor de Internet. O procedimento de implementação, os cenários e as métricas de avaliação são apresentados no capítulo seguinte.

3 METODOLOGIA

Este capítulo apresenta o método proposto para comparar uma arquitetura de agente único e uma arquitetura multiagente em atendimentos textuais de um provedor de Internet. A comparação será realizada em condições controladas, utilizando os mesmos cenários conversacionais, o mesmo modelo de linguagem, os mesmos parâmetros de geração e a mesma base de conhecimento. Serão observadas a qualidade das respostas, a conclusão das tarefas, os encaminhamentos realizados e os recursos consumidos durante cada execução.

3.1 NATUREZA DO TRABALHO

O trabalho possui natureza aplicada, abordagem quantitativa e procedimento experimental. A pesquisa é aplicada porque envolve a especificação e a implementação de dois protótipos destinados a uma situação de atendimento. O procedimento experimental será utilizado para comparar as arquiteturas sob condições controladas.

Não se pretende desenvolver um sistema comercial completo. Os protótipos serão utilizados como instrumentos de experimentação, com as funções necessárias para receber mensagens, manter o estado da conversa, consultar a base de conhecimento, produzir respostas e registrar informações sobre cada execução.

A etapa de TCC1 compreende a fundamentação teórica, a especificação das arquiteturas, a definição preliminar dos cenários e das métricas e a verificação de viabilidade por meio de um protótipo exploratório. A implementação definitiva, a execução dos casos de teste e a análise dos resultados serão realizadas no TCC2.

3.2 DELINEAMENTO EXPERIMENTAL

O fator de comparação será a arquitetura utilizada no atendimento, com dois níveis: agente único e sistema multiagente. As duas soluções receberão cenários equivalentes e serão submetidas às mesmas condições de execução.

A variável independente será a arquitetura. As variáveis dependentes serão as medidas de qualidade, encaminhamento e consumo obtidas durante cada conversa. Serão controlados o modelo de linguagem, os parâmetros de geração, a base de conhecimento, as regras de atendimento, os cenários, a ordem das mensagens e o ambiente de execução.

Cada cenário será executado uma vez em cada arquitetura. Não serão realizadas repetições de uma mesma combinação de cenário e arquitetura. As diferentes formas de expressão de uma intenção serão representadas por variações de redação previstas no

próprio conjunto de testes.

3.2.1 Arquiteturas avaliadas

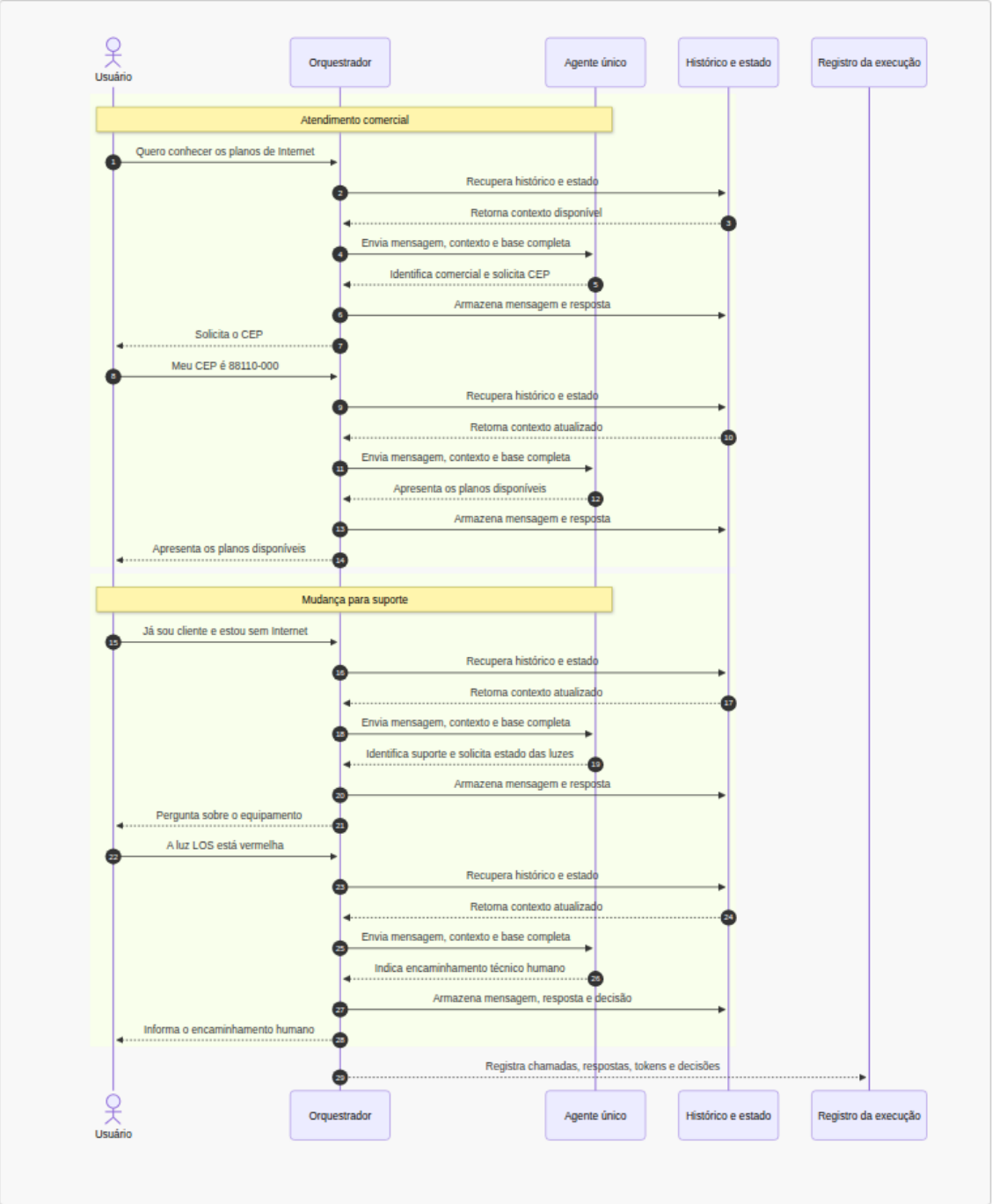
3.2.1.1 Arquitetura de agente único

Na primeira arquitetura, um único agente receberá as instruções e a base de conhecimento referentes às quatro áreas consideradas no estudo: primeiro atendimento, comercial, ouvidoria e suporte técnico. Para cada mensagem, o agente deverá interpretar a solicitação, considerar o histórico da conversa, identificar o assunto e produzir a resposta correspondente.

Um componente externo de orquestração será responsável por receber as mensagens, manter o histórico e os fatos relevantes, acionar o agente e registrar os resultados. As decisões de classificação e de conteúdo das respostas serão produzidas pelo agente único. O orquestrador organizará a execução e verificará a consistência das saídas estruturadas.

A Figura 1 apresenta um exemplo de atendimento que começa com uma consulta comercial e passa para uma solicitação de suporte técnico.

Figura 1 – Arquitetura proposta de agente único



Nota: O mesmo agente utiliza o histórico da conversa, identifica as mudanças de assunto e aplica as instruções das quatro áreas de atendimento. Não há transferência entre agentes de inteligência artificial.

Fonte: Elaborada pelo autor.

A cada mensagem, o orquestrador preparará a entrada do agente, reunindo as instruções de atendimento, a base de conhecimento completa, o histórico e os fatos relevantes disponíveis. No primeiro turno, o histórico estará vazio. Nos turnos seguintes, conterá as mensagens e respostas anteriores. Os fatos relevantes complementarão esse histórico, sem substituí-lo.

No exemplo, a solicitação de planos será identificada como comercial e resultará em uma pergunta sobre o CEP. A classificação da área não representará a ativação de outro agente: o mesmo componente continuará responsável pelo atendimento. Quando o usuário relatar falta de Internet, o agente deverá reconhecer a nova necessidade e aplicar as regras de suporte técnico.

O Código 3.1 apresenta um exemplo ilustrativo da saída estruturada utilizada no protótipo exploratório. Os nomes e a organização dos campos poderão ser refinados durante a implementação definitiva no TCC2.

Código 3.1 – Exemplo ilustrativo de retorno do agente único

```
{
  "response": "Para informar os planos disponíveis, qual é seu CEP?",
  "intent": "comercial_planos",
  "active_area": "commercial",
  "action": "respond",
  "handoff_target": null,
  "context_facts": [
    "O usuário deseja conhecer os planos."
  ]
}
```

O campo `response` contém o texto apresentado ao usuário. Os campos `intent` e `active_area` registram a intenção e a área identificadas. Os campos `action` e `handoff_target` representam a ação proposta e o eventual destino de atendimento humano. O campo `context_facts` contém a lista atualizada de fatos relevantes para a continuidade da conversa.

Após receber a saída, o orquestrador atualizará o histórico e os fatos relevantes, registrará as informações da execução e apresentará ao usuário o conteúdo de `response`. A indicação de atendimento humano constituirá uma decisão registrada pelo protótipo, sem transferência operacional para um sistema externo.

No diagrama, o histórico e o estado são apresentados separadamente para explicitar sua função. Essa representação não pressupõe a utilização de um serviço ou banco de dados independente. Da mesma forma, a seta final de registro sintetiza as informações coletadas ao longo das chamadas e reunidas no resultado da execução.

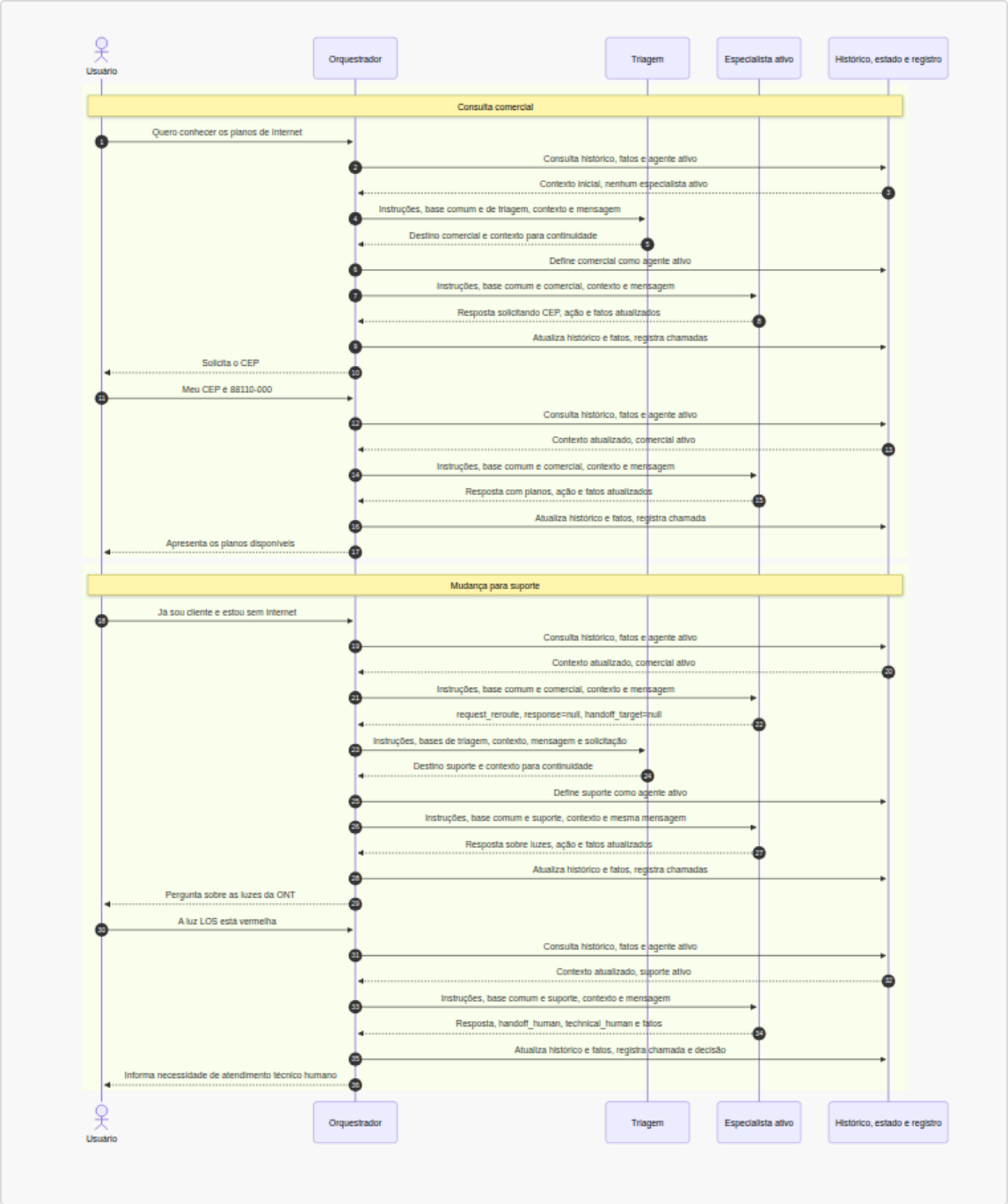
3.2.1.2 Arquitetura multiagente

Na segunda arquitetura, as responsabilidades serão distribuídas entre um agente de triagem e quatro agentes especializados: primeiro atendimento, comercial, ouvidoria e suporte técnico. Os componentes utilizarão o mesmo modelo de linguagem, com instruções e bases de conhecimento organizadas conforme suas responsabilidades.

O orquestrador manterá o histórico, os fatos relevantes e a identificação do especialista ativo. A triagem será acionada no início da conversa e quando o especialista solicitar uma nova classificação. Os agentes devolverão suas saídas ao orquestrador, sem acionar diretamente outros componentes.

A Figura 2 apresenta um exemplo que começa com uma consulta comercial e passa para uma solicitação de suporte técnico.

Figura 2 – Arquitetura multiagente proposta



Nota: O participante especialista ativo representa o comercial inicialmente e o suporte após a nova triagem. O histórico, o estado e o registro estão agrupados para simplificar a apresentação, sem pressupor um banco de dados ou serviço independente.

Fonte: Elaborada pelo autor.

Na primeira mensagem, ainda não haverá especialista ativo. O orquestrador acionará a triagem, fornecendo suas instruções, a base comum, as regras de classificação e o contexto disponível. A triagem identificará a área responsável e devolverá uma saída estruturada com o destino, a intenção, a justificativa e as informações pertinentes à continuidade do atendimento.

O Código 3.2 apresenta um exemplo ilustrativo desse retorno no protótipo exploratório.

Código 3.2 – Exemplo ilustrativo de retorno da triagem

```
{
  "destination": "commercial",
  "intent": "comercial_planos",
  "reason": "O usuário deseja conhecer os planos de Internet.",
  "context_to_forward": [
    "O usuário deseja conhecer os planos."
  ]
}
```

O campo `destination` indicará o especialista a ser acionado pelo orquestrador. O campo `context_to_forward` fornecerá informações para a continuidade, complementando o histórico completo. A justificativa e a classificação serão registros internos, sem apresentação direta ao usuário.

Após a seleção, o especialista receberá suas instruções, a base comum acrescida da base de sua área, o histórico, os fatos relevantes e a mensagem atual. No exemplo, o comercial solicitará o CEP e permanecerá ativo para processar a mensagem seguinte, sem nova triagem.

Quando o usuário relatar falta de Internet, a mensagem será inicialmente processada pelo comercial, pois esse ainda será o especialista ativo. Ao identificar uma necessidade fora de sua área, ele deverá solicitar nova classificação, conforme o exemplo apresentado no Código 3.3.

A ação `request_reroute` solicitará uma nova triagem. Nesse caso, `response` e `handoff_target` deverão ser nulos, pois o especialista não produzirá uma resposta ao usuário nem indicará atendimento humano nessa etapa.

O orquestrador fornecerá à triagem a mensagem atual, o contexto e a solicitação do especialista anterior. Após receber o novo destino, atualizará o especialista ativo e acionará o componente selecionado para processar a mesma mensagem. No exemplo, o suporte solicitará o estado das luzes da ONT.

Ao receber a informação de luz LOS vermelha, o suporte aplicará a regra correspondente e indicará a necessidade de atendimento técnico humano. Essa decisão será

Código 3.3 – Exemplo ilustrativo de solicitação de nova triagem

```
{
  "response": null,
  "intent": "suporte_sem_conexao",
  "action": "request_reroute",
  "handoff_target": null,
  "context_facts": [
    "O usuário informou o CEP 88110-000.",
    "O usuário informa que já é cliente.",
    "O usuário relata falta de Internet."
  ]
}
```

registrada como `handoff_human`, com destino `technical_human`. Ela não representará uma transferência operacional para um sistema externo.

O exemplo contém quatro turnos do usuário e sete chamadas ao modelo: duas no primeiro turno, uma no segundo, três na mudança de assunto e uma no último turno. Haverá uma troca entre especialistas, do comercial para o suporte; a seleção inicial não será contada como troca.

O histórico preservará as mensagens do usuário e as respostas finais do assistente. As saídas intermediárias e as medições de cada chamada serão mantidas nos registros da execução. Os formatos apresentados correspondem ao protótipo exploratório e poderão ser refinados durante a implementação definitiva no TCC2.

3.2.2 Variáveis controladas

Para reduzir diferenças que não estejam relacionadas à arquitetura, serão mantidos constantes:

- o modelo de linguagem e sua versão;
- os parâmetros de geração;
- a base total de conhecimento;
- as regras de atendimento;
- os casos de teste;
- a ordem das mensagens em cada cenário;
- o modelo utilizado como avaliador;
- os limiares e as configurações das métricas;
- o ambiente de execução e as versões das bibliotecas.

Os prompts serão diferentes somente no que for necessário para representar as responsabilidades de cada arquitetura. No agente único, todas as instruções ficarão reunidas em um mesmo contexto. No sistema multiagente, as instruções serão distribuídas entre a triagem e os agentes especializados.

3.2.3 Variáveis avaliadas

As variáveis avaliadas serão:

- correção e relevância das respostas;
- retenção das informações fornecidas durante a conversa;
- conclusão da tarefa;
- correspondência entre o encaminhamento realizado e o esperado;
- quantidade de trocas entre agentes;
- número de chamadas ao modelo;
- quantidade de tokens de entrada e saída;
- custo estimado de execução.

3.3 BASE DE CONHECIMENTO E PROMPTS

A base de conhecimento reunirá informações fictícias sobre serviços, procedimentos de atendimento e condições de encaminhamento. O conteúdo será organizado em uma parte comum e em partes específicas para primeiro atendimento, comercial, ouvidoria e suporte técnico.

As duas arquiteturas terão acesso ao mesmo conteúdo total. No agente único, as partes serão reunidas no mesmo contexto. Na arquitetura multiagente, o conteúdo será distribuído de acordo com a responsabilidade de cada agente. Não será implementado um mecanismo de geração aumentada por recuperação, pois os documentos serão carregados diretamente no contexto dos agentes.

Os prompts definirão a função de cada agente, as informações disponíveis, as ações permitidas e o formato esperado da saída. Sempre que houver uma decisão de encaminhamento, a saída deverá ser estruturada para permitir seu registro e sua verificação automática.

Os prompts serão versionados e permanecerão inalterados durante o experimento principal. Alterações realizadas durante a preparação ou o teste-piloto serão registradas antes da execução dos casos definitivos.

3.4 CENÁRIOS CONVERSACIONAIS

Os testes serão formados por conversas com múltiplos turnos, nas quais as informações serão apresentadas gradualmente. Cada cenário conterá:

- as mensagens do usuário;
- o assunto esperado em cada etapa;
- o agente ou fila esperada;
- as informações que deverão permanecer no contexto;
- a resposta ou o comportamento esperado;
- os critérios necessários para considerar a tarefa concluída.

Serão preparados 48 cenários sintéticos, distribuídos igualmente entre as quatro áreas de atendimento. Para cada área, serão considerados três tipos de situação, cada um com quatro variações de redação. Dessa forma, cada área possuirá 12 cenários.

Tabela 1 – Distribuição prevista dos cenários conversacionais

Área	Situações	Variações	Total
Primeiro atendimento	3	4	12
Comercial	3	4	12
Ouvidoria	3	4	12
Suporte técnico	3	4	12
Total	–	–	48

Fonte: Elaborada pelo autor.

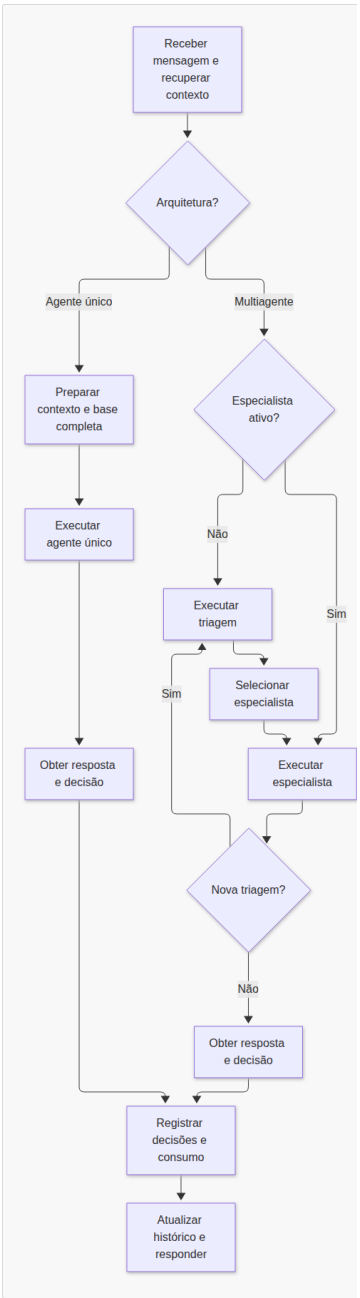
Os casos utilizarão nomes, documentos, endereços, números de telefone e protocolos fictícios. Não haverá interação com clientes reais.

O DeepEval poderá ser utilizado como apoio para sugerir variações de redação ou simular turnos de uma conversa (CONFIDENT AI, 2026). Entretanto, cada cenário será revisado antes de ser incorporado ao conjunto definitivo. A revisão verificará a clareza das mensagens, o destino esperado, os fatos necessários e a equivalência de aplicação entre as duas arquiteturas.

3.4.1 Cenário conversacional de exemplo

O cenário inicial começará com uma solicitação comercial e, durante a mesma conversa, passará a tratar de uma falha de conexão e de uma reclamação. Esse caso permitirá observar a preservação do contexto e as decisões de encaminhamento.

Figura 3 – Fluxo de processamento de uma mensagem



Nota: A arquitetura de agente único realiza uma chamada principal ao modelo por mensagem. Na arquitetura multiagente, a triagem é executada quando não existe agente ativo ou quando o especialista solicita nova classificação após identificar uma mudança de assunto.

Fonte: Elaborada pelo autor.

Tabela 2 – Exemplo de cenário conversacional

Turno	Mensagem do usuário	Agente esperado
1	Quero conhecer os planos de Internet.	Comercial
2	Meu CEP é 88110-000.	Comercial
3	Na verdade, já sou cliente e estou sem Internet.	Suporte técnico
4	A luz LOS está vermelha.	Suporte técnico
5	Isso já aconteceu outras vezes e quero reclamar.	Ouvidoria

Fonte: Elaborada pelo autor.

3.5 MATERIAIS E FERRAMENTAS

Os protótipos serão desenvolvidos em Python. Os cenários, as configurações e os resultados serão armazenados em arquivos estruturados no formato *JavaScript Object Notation* (JSON). O código-fonte, os prompts, a base de conhecimento e os documentos auxiliares serão versionados com Git.

O acesso ao modelo de linguagem será realizado por meio de uma *Application Programming Interface* (API). Será utilizado o modelo GPT-4.1-mini, mantido igual nas duas arquiteturas. Esse modelo foi selecionado por oferecer capacidade de interpretação de instruções e geração de respostas com custo compatível com o volume previsto de execuções (OPENAI, 2026). O identificador exato do modelo retornado pela API, os parâmetros de geração e a data de execução serão registrados para permitir a reprodução do experimento.

O DeepEval será utilizado na organização dos casos conversacionais e na aplicação de métricas relacionadas à relevância, retenção de conhecimento e conclusão das tarefas (CONFIDENT AI, 2026). Verificações objetivas, como fila escolhida, número de chamadas e consumo de tokens, serão realizadas diretamente por scripts desenvolvidos em Python.

Planilhas ou scripts de análise poderão ser utilizados para consolidar os resultados e produzir tabelas, matrizes de confusão e gráficos comparativos.

A implementação e a execução dos scripts serão realizadas em computador pessoal, com ambiente Python e acesso à Internet. Como o processamento do modelo de linguagem será realizado por meio de uma API, não será necessária uma unidade de processamento gráfico local.

Antes do teste-piloto, serão verificadas a instalação das bibliotecas, a configuração das credenciais e a disponibilidade de acesso aos modelos utilizados no atendimento e na avaliação. As versões das dependências serão registradas junto ao código-fonte.

O orçamento necessário será estimado a partir do consumo observado no teste-

piloto e do número previsto de execuções. Essa estimativa considerará separadamente os custos dos protótipos e os custos do modelo avaliador utilizado pelo DeepEval.

3.6 PROCEDIMENTO EXPERIMENTAL

O experimento será realizado de acordo com as seguintes etapas:

1. finalizar e revisar a base de conhecimento;
2. finalizar os prompts das duas arquiteturas;
3. implementar o agente único, o orquestrador, a triagem e os agentes especializados;
4. elaborar e revisar os 48 cenários conversacionais;
5. executar um teste-piloto para verificar o formato das entradas, das saídas e dos registros;
6. congelar as versões dos prompts, dos cenários, do modelo e das configurações;
7. executar cada cenário uma vez em cada arquitetura;
8. aplicar as métricas automáticas e as verificações determinísticas;
9. revisar manualmente uma amostra das avaliações;
10. consolidar e comparar os resultados.

Quadro 2 – Correspondência entre objetivos específicos, etapas e entregas previstas

Objetivo específico	**Etapas**	**Entrega prevista**
Definir a base de conhecimento e as regras comuns	1 e 2	Base de conhecimento e prompts revisados e versionados.
Especificar e implementar os dois protótipos	3	Protótipos funcionais de agente único e multiagente.
Elaborar e revisar os cenários conversacionais	4	Conjunto de 48 cenários com comportamentos esperados e critérios de conclusão.
Avaliar respostas, conclusão das tarefas e precisão dos encaminhamentos	7, 8 e 9	Avaliações automáticas, verificações determinísticas e revisão manual de uma amostra.
Medir tokens e chamadas ao modelo	7 e 8	Registros de consumo por conversa e arquitetura.
Estimar o custo de execução	10	Estimativas de custo calculadas a partir do consumo registrado.
Comparar os resultados das arquiteturas	10	Tabelas, gráficos e análise das diferenças observadas.

Fonte: Elaborada pelo autor.

As etapas 5 e 6 correspondem à validação do procedimento no teste-piloto e ao congelamento das configurações. Essas etapas apoiam os objetivos de avaliação e comparação, assegurando que as duas arquiteturas sejam submetidas às condições previamente estabelecidas.

Os 48 cenários serão executados uma vez em cada arquitetura, totalizando 96 execuções. Cada execução corresponderá a uma conversa completa, e não a uma mensagem isolada.

As mensagens serão enviadas na ordem definida no cenário. Antes de cada nova mensagem, a resposta anterior será armazenada no histórico. Ao final da conversa, serão registrados os agentes acionados, as decisões de encaminhamento, as respostas produzidas, os tokens consumidos e o número de chamadas ao modelo.

O teste-piloto será utilizado para verificar o funcionamento do procedimento, identificar ambiguidades nos cenários, revisar os critérios e limiares de avaliação e estimar o orçamento necessário para o experimento principal. Seus resultados não serão incluídos na comparação final.

3.7 MÉTRICAS E CRITÉRIOS DE AVALIAÇÃO

A qualidade das conversas será avaliada por métricas automáticas e verificações determinísticas.

As métricas conversacionais do DeepEval inicialmente previstas são:

- relevância dos turnos;
- retenção de conhecimento;
- completude da conversa.

As mesmas configurações, o mesmo modelo avaliador e os mesmos limiares serão utilizados nas duas arquiteturas. Uma amostra das avaliações será revisada manualmente para verificar se as pontuações e justificativas correspondem aos critérios definidos.

A identificação da área de atendimento será avaliada pela comparação entre a área indicada pelo sistema e a área esperada em cada turno. Na arquitetura de agente único, o agente registrará a área identificada em sua saída estruturada, sem realizar uma troca interna de agente. Na arquitetura multiagente, será registrada a área correspondente ao especialista acionado para responder à mensagem. Serão calculadas a acurácia geral e a acurácia por área. Uma matriz de confusão será utilizada para identificar quais áreas foram confundidas com maior frequência.

As trocas internas entre especialistas serão contabilizadas separadamente e somente na arquitetura multiagente. As transferências para atendimento humano serão avaliadas nas duas arquiteturas, considerando se ocorreram nas situações previstas pelas regras e pelos critérios de cada cenário.

O consumo será medido pelo número de chamadas e pelas quantidades de tokens de entrada e saída. O custo de uma execução será estimado conforme os preços vigentes do modelo na data do experimento:

$$C = T_e P_e + T_s P_s, \tag{3.1}$$

em que T_e representa a quantidade de tokens de entrada, T_s a quantidade de tokens de saída, P_e o preço por token de entrada e P_s o preço por token de saída.

Os preços não serão fixados antecipadamente no texto, pois poderão sofrer alterações. A fonte e a data da consulta serão registradas no momento da execução.

Para cada cenário, serão definidos previamente os comportamentos obrigatórios, as informações que deverão ser preservadas e as condições necessárias para a conclusão da tarefa. A correção das respostas será verificada pela sua conformidade com a base de conhecimento e com as regras de atendimento, incluindo a ausência de informações inventadas ou de ações não permitidas.

Uma tarefa será considerada concluída quando o sistema cumprir as condições previstas no cenário. Nos casos que exigirem atendimento humano, o encaminhamento adequado, acompanhado das informações necessárias à continuidade do atendimento, poderá constituir a conclusão esperada.

Os limiares das métricas automáticas serão definidos e documentados durante o teste-piloto, antes da execução do experimento principal, e permanecerão iguais para as duas arquiteturas. Esses limiares serão utilizados para avaliar o atendimento aos critérios definidos, enquanto as pontuações serão mantidas para a análise comparativa.

O sucesso da pesquisa será caracterizado pela realização da comparação sob as condições estabelecidas e pela identificação das diferenças de qualidade e consumo de recursos observadas nos cenários avaliados, independentemente de uma arquitetura superar a outra.

3.8 APRESENTAÇÃO PREVISTA DOS RESULTADOS

Os resultados serão organizados conforme a estrutura apresentada na Tabela 3. Os campos serão preenchidos depois da execução do experimento no TCC2.

A identificação da área de atendimento e a adequação das transferências para atendimento humano serão avaliadas nas duas arquiteturas. As trocas internas entre especi-

Tabela 3 – Estrutura prevista para apresentação dos resultados

Métrica	Agente único	Multiagente
Relevância dos turnos	–	–
Retenção de conhecimento	–	–
Completude da conversa	–	–
Acurácia da identificação da área	–	–
Adequação das transferências para humanos	–	–
Trocas internas entre especialistas	Não se aplica	–
Número de chamadas ao modelo	–	–
Tokens de entrada	–	–
Tokens de saída	–	–
Custo estimado	–	–

Fonte: Elaborada pelo autor.

alistas serão apresentadas somente para a arquitetura multiagente, pois esse mecanismo não se aplica ao agente único.

3.9 PRIVACIDADE E PROTEÇÃO DOS DADOS

Os cenários serão produzidos especificamente para o estudo, com base em situações típicas de atendimento de provedores de Internet. Não serão utilizadas conversas, cadastros ou registros de clientes reais.

Os nomes, documentos, endereços, números de telefone, protocolos e demais informações presentes nos testes serão fictícios. A experiência profissional do autor será utilizada somente como referência para a elaboração das situações, sem reprodução de casos que permitam identificar clientes, funcionários ou empresas.

Dessa forma, não serão enviados dados pessoais reais ao modelo de linguagem ou às ferramentas utilizadas no experimento.

3.10 RISCOS E LIMITAÇÕES

Entre os riscos previstos estão a variação das respostas do modelo, alterações ou indisponibilidade da API, mudanças na versão do modelo, diferenças excessivas entre os prompts e limitações da avaliação automática de respostas abertas.

A execução única de cada combinação de cenário e arquitetura não permitirá medir a variação de uma mesma resposta em execuções repetidas. Para reduzir essa limitação, o conjunto de testes utilizará diferentes cenários e variações de redação, aplicados igualmente às duas arquiteturas.

Também existe o risco de uma métrica baseada em modelo atribuir uma pontuação inadequada. Como medidas de controle, as duas arquiteturas serão avaliadas com as

mesmas configurações, os encaminhamentos serão verificados por regras determinísticas e uma amostra das respostas será revisada manualmente.

Os resultados ficarão limitados ao modelo, à base de conhecimento, aos prompts e aos cenários definidos neste trabalho. Portanto, não poderão ser generalizados diretamente para todos os modelos de linguagem ou sistemas comerciais de atendimento.

Em caso de indisponibilidade temporária da API, as execuções afetadas serão interrompidas e reagendadas. Tentativas incompletas ou com falhas de comunicação serão registradas e não serão consideradas conversas válidas na comparação. Seu consumo, quando houver, será contabilizado separadamente.

Se o modelo ou a versão adotada deixar de estar disponível, será selecionada uma alternativa compatível com os recursos do projeto. Nesse caso, o teste-piloto será refeito e as duas arquiteturas serão avaliadas com o mesmo modelo e as mesmas configurações. Resultados obtidos com versões diferentes não serão reunidos na mesma comparação.

Caso uma métrica automática apresente incompatibilidade técnica ou avaliações inconsistentes no teste-piloto, seu procedimento será revisado antes do experimento principal. Se necessário, essa métrica será substituída por uma avaliação manual baseada em critérios previamente documentados e aplicada igualmente às duas arquiteturas. A alteração e suas limitações serão registradas no trabalho.

4 CRONOGRAMA DE ATIVIDADES

As atividades previstas para o TCC2 serão desenvolvidas no primeiro semestre de 2027. O cronograma contempla a implementação dos protótipos, a preparação dos cenários conversacionais, a execução do experimento, a análise dos resultados e a conclusão da monografia.

A redação e as revisões com o orientador ocorrerão ao longo do semestre. Os períodos previstos para a defesa e a entrega final serão ajustados conforme o calendário oficial da unidade curricular.

Tabela 4 – Cronograma previsto para o TCC2 no primeiro semestre de 2027

Atividade	**Fev.**	**Mar.**	**Abr.**	**Maio**	**Jun.**	**Jul.**
Revisão da base de conhecimento e dos prompts	X	X				
Implementação dos protótipos e dos registros de execução	X	X				
Elaboração e revisão dos cenários conversacionais	X	X				
Integração das métricas e execução do teste-piloto		X	X			
Congelamento das configurações e execução do experimento			X			
Consolidação e análise comparativa dos resultados			X	X		
Redação da monografia e revisões com o orientador	X	X	X	X	X	
Preparação da apresentação e defesa pública					X	
Correções solicitadas pela banca e entrega final					X	X

Fonte: Elaborada pelo autor.

A execução do experimento será iniciada após a conclusão do teste-piloto e o congelamento dos prompts, dos cenários e das configurações. A análise utilizará os registros obtidos nessa execução. Após a defesa, serão realizadas as correções solicitadas pela banca e a entrega da versão final.

REFERÊNCIAS

AMRI, M. F. et al. Optimizing LLM Chatbot to Human Handover with Multi-Agent AI Approach. In: 2026 International Conference on Advancement in Data Science, E-learning and Information System (ICADEIS). Jun. 2026. P. 1–6. DOI: 10.1109/ICADEIS71120.2026.11644567.

CONFIDENT AI. **DeepEval Documentation**. 2026. Disponível em: <https://deepeval.com/docs/introduction>. Acesso em: 8 out. 2026.

LEE, K.-Y.; TSENG, P.-A.; CHEN, W.-C. Enhancing Telecom Operation Support Systems with Multi-Agent Large Language Models. In: 2025 11th International Conference on Computing and Artificial Intelligence (ICCAI). Mar. 2025. P. 309–316. DOI: 10.1109/ICCAI66501.2025.00055.

MONCADA, C. et al. Large Language Models and Multi-Agent Systems for Customer Support Ticket Automation. In: 2025 IEEE Ninth Ecuador Technical Chapters Meeting (ETCM). Out. 2025. P. 1–6. DOI: 10.1109/ETCM67548.2025.11304425.

OPENAI. **GPT-4.1 mini Model**. 2026. Disponível em: <https://developers.openai.com/api/docs/models/gpt-4.1-mini>. Acesso em: 10 out. 2026.

OWOTOGBE, Joshua. Assessing and Enhancing the Robustness of LLM-Based Multi-Agent Systems Through Chaos Engineering. In: 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN). Abr. 2025. P. 250–252. DOI: 10.1109/CAIN66642.2025.00039.

YAN, Bingyu et al. Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems. **Frontiers of Computer Science**, 2026. DOI: 10.1007/s11704-026-50857-y. Disponível em: <https://arxiv.org/abs/2502.14321>. Acesso em: 25 set. 2026.

YANG, Wangying; XU, Peng; ZHANG, Xiaogong. A Two-Stage Intent Recognition Framework in Multi-Agent Environments. In: 2025 6th International Conference on Computer Engineering and Application (ICCEA). Abr. 2025. P. 1–5. DOI: 10.1109/ICCEA65460.2025.11103313.